V K PARTHIP
VML24CS246
S4 CSE D, ROLL NO:61

ASSIGNMENT - 1

QUESTION - 8

Write a C program rr.c that implements Round Robin CPU scheduling.

InputFormat:
n
PID1 AT1 BT1
PID2 AT2 BT2
.....
PIDn ATn BTn
q
Sample Input:
4
P1 0 5
P2 1 4
P3 2 2
P4 3 1
2
Sample Output:
Waiting Time:
P1 7
P2 6
P3 2
P4 5
Turnaround Time:
P1 12
P2 10
P3 4
P4 6
Average Waiting Time: 5.00
Average Turnaround Time: 8.00

PROGRAM:

```c
#include <stdio.h>

struct process {
    char pid[10];
    int at, bt;
    int ct, tat, wt, rt;
};

void sorting(struct process p[], int n) {
    for(int i = 0; i < n-1; i++)
        for(int j = i+1; j < n; j++)
            if(p[i].at > p[j].at) {
                struct process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
}

int main() {

    int n, quantum;
    struct process p[20];
```

```c
int main() {

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter PID, Arrival Time and Burst Time for each process:\n");
    for(int i = 0; i < n; i++)
        scanf("%s %d %d", p[i].pid, &p[i].at, &p[i].bt);

    printf("Enter quantum time: ");
    scanf("%d", &quantum);

    sorting(p, n);

    int q[100], front = 0, rear = 0;
    int visited[20] = {0};
    int time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;

    for(int i = 0; i < n; i++)
        p[i].rt = p[i].bt;
```

rr.c

C/C++: gcc build and...

rr.c > ...

```c
int main() {
    time = p[0].at;
    q[rear++] = 0;
    visited[0] = 1;

    while(completed < n) {

        int i = q[front++];

        if(p[i].rt > quantum) {
            time += quantum;
            p[i].rt -= quantum;
        } else {
            time += p[i].rt;
            p[i].rt = 0;
            p[i].ct = time;
            p[i].tat = p[i].ct - p[i].at;
            p[i].wt = p[i].tat - p[i].bt;
            completed++;
        }

        for(int j = 0; j < n; j++) {
```

rr.c

C/C++: gcc build and...

rr.c > ...

```c
int main() {
    while(completed < n) {
        for(int j = 0; j < n; j++) {
            if(p[j].at <= time && visited[j] == 0) {
                q[rear++] = j;
                visited[j] = 1;
            }
        }

        if(p[i].rt > 0)
            q[rear++] = i;
    }

    printf("\nWaiting Time:\n");
    for(int i = 0; i < n; i++) {
        printf("%s %d\n", p[i].pid, p[i].wt);
        total_wt += p[i].wt;
    }

    printf("Turnaround Time:\n");
    for(int i = 0; i < n; i++) {
        printf("%s %d\n", p[i].pid, p[i].tat);
```

rr.c

C/C++: gcc build and...

rr.c > ...

```c
int main() {
    for(int i = 0; i < n; i++) {
        total_tat += p[i].tat;
    }

    printf("Average Waiting Time: %.2f\n", total_wt/n);
    printf("Average Turnaround Time: %.2f\n", total_tat/n);

    return 0;
}

```

OUTPUT:

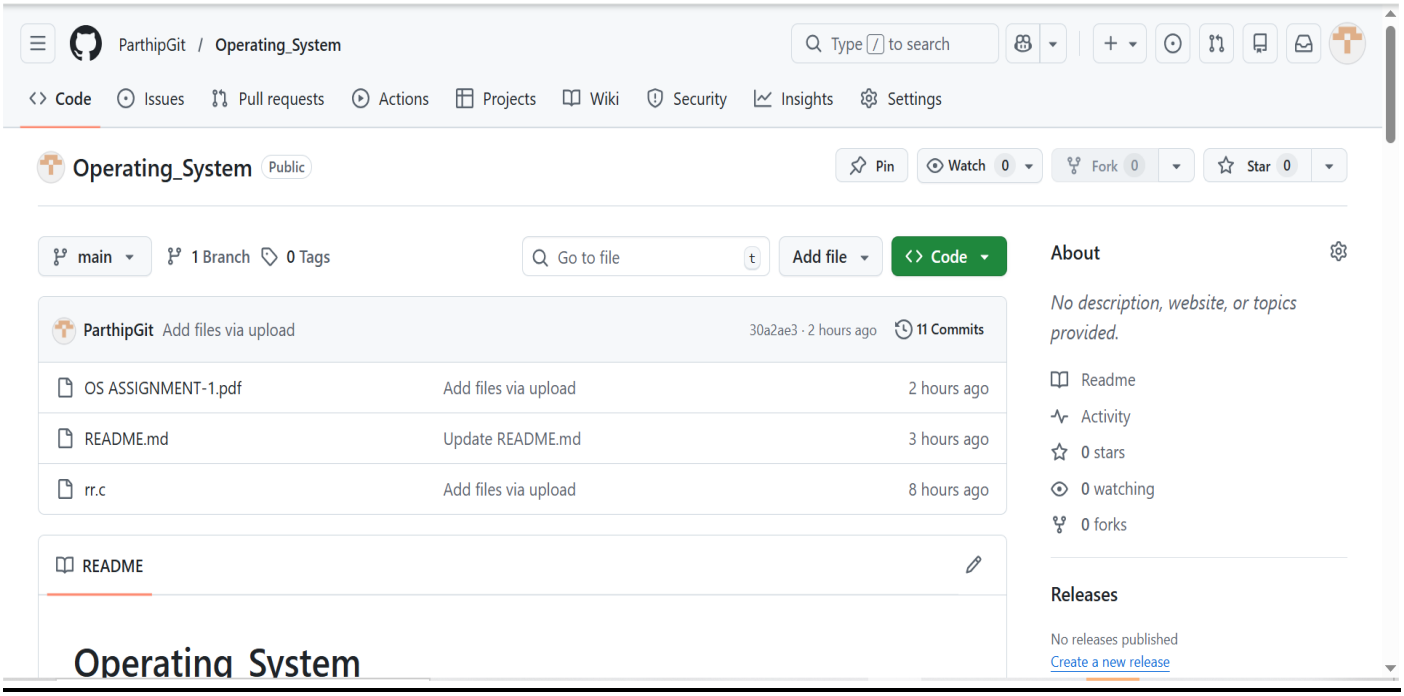
```
rr.c
C/C++: gcc build and...
rr.c > ...
19    int main() {

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter number of processes: 4
Enter PID, Arrival Time and Burst Time for each process:
P1 0 5
P2 1 4
P3 2 2
P4 3 1
Enter quantum time: 2

Waiting Time:
P1 7
P2 6
P3 2
P4 5
Turnaround Time:
P1 12
P2 10
P3 4
P4 6
Average Waiting Time: 5.00
Average Turnaround Time: 8.00
[1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm}

bash
C/C++: ...
cppdbg: rr
cppdbg: rr
```

GITHUB:

LINK: https://github.com/ParthipGit/Operating_System.git